Technical Interview Questions - Part 3 - SQL

Giuliana Olmos · 7 de noviembre de 2024 · https://dev.to/giulianaolmos/technical-interview-questions-part-3-sql-ni4

Introduction

Hi everyone! 🤗 I’m back with a new set of SQL questions. They aren't too complicated (I think), but I've often struggled to explain them clearly 🥹. I could give examples from my own use, but I couldn't always find the right words to describe them.

I'll keep this intro short, so let's dive into the questions! 🤓

Questions

1. What is a key in SQL, and what types do you know?
2. What is a subquery?
3. What is a transaction in SQL??

Question 1: What is a key in SQL, and what types do you know?

The short answer is...

> A key is a set of attributes that we use to identify a specific row in a table or to create relationships between tables.

> There are different types of keys:

- Primary Key
- Candidate Key
- Super Key
- Alternate Key
- Unique Key
- Composite Key
- Foreign Key

...

Primary Key

A **primary key** is the key to uniquely identifying each row (record) in a table.

- A table can have only **one** primary key.
- Can be made up of **one or more** columns.
- Cannot be **NULL**.
- Cannot be duplicated.

Table `students`

StudentID	FirstName	LastName
1	John	Doe
2	Jane	Smith
3	Bob	Brown

In this example, `StudentID` is the primary key. Each student has only one `StudentID`.

...

Candidate key

A **candidate key** is a set of columns that can uniquely identify each row in a table. A table can have multiple candidate keys, but only **one** will be chosen as the **primary key**. A candidate key must be **minimal** and cannot include extra columns.

Table `vehicles`

VIN	LicensePlate	Model
1HGCM82633A123456	ABC123	Civic
1HGCM82633A654321	XYZ987	Accord

`VIN` and `LicensePlate` are the candidate keys for this table because both can uniquely identify a vehicle.

...

Super Key

A **super key** is a set of columns that can uniquely identify records in a table.
All **candidate keys** are super keys, but not all super keys are candidate keys because super keys may include columns that are not necessary for uniqueness.

Table `employees`

EmployeeID	Email	PhoneNumber	Name
1	john.doe@example.com	123-456-7890	John Doe
2	jane.smith@example.com	987-654-3210	Jane Smith

- `{EmployeeID, Email}` and `{EmployeeID, Email, PhoneNumber}` are **Super Keys** because the combination of those columns can uniquely identify each record.

...

Alternate Key

When a primary key is selected, any remaining key becomes an alternate key.

Table `Authors`

AuthorID	SSN	Name
1	123-45-6789	John Doe
2	987-65-4321	Jane Smith

Primary Key => `AuthorID` **Alternate Key** => `SSN` (Since it can also uniquely identify a record but is not chosen as the Primary Key).

...

Unique Key

It also uniquely identifies records BUT allows **NULL** values.

Table `employees`

EmployeeID	Email	Name
101	john.doe@example.com	John Doe
102	jane.smith@example.com	Jane Smith
103	bob.brown@example.com	Bob Brown

Primary Key => `EmployeeID` (Unique for each employee). **Unique Key** => `Email` (Each email is unique).

...

Composite Key

These keys are made up of **two or more columns** that uniquely identify a record.

Table `CourseEnrollments`

StudentID	CourseID	EnrollmentDate
1	101	2024-01-01
2	102	2024-01-02
1	102	2024-01-03

The **composite key** is `{StudentID, CourseID}` because a student can enroll in multiple courses, but this combination is unique.

...

Foreign Key

A **foreign key** is used to set relationships between tables. It ensures **referential integrity** between tables, helping to avoid duplicate data.

For this example, I need two tables:

Table `orders`

OrderID	CustomerID	OrderDate
1	101	2024-01-01
2	102	2024-01-02
3	101	2024-01-03

Table `customers`

CustomerID	CustomerName
101	John Doe
102	Jane Smith

The **primary keys** are `OrderID` for the `Orders` table and `CustomerID` for the `Customers` table. The **foreign key** in `Orders`: `CustomerID` (References `CustomerID` in `Customers`).

Question 2: What is a subquery?

Answer:

> A **subquery** is a query that appears inside another query statement. It's also called an `inner query` or `nested query`. It's used to perform a query that returns data to be used by the `"outer"` query.

An example: If I have two tables...

Table `Employees`

EmployeeID	Name	DepartmentID
1	John Doe	10
2	Jane Smith	20
3	Bob Brown	10
4	Alice White	30

And the table `Departments`

DepartmentID	DepartmentName
10	HR
20	Finance
30	IT

If I want to find the names of employees in the HR department. The query I should write looks like this:

```
SELECT Name
FROM Employees
WHERE DepartmentID = (
    SELECT DepartmentID
    FROM Departments
    WHERE DepartmentName = 'HR'
);
```

Where the `inner query` is

```
SELECT DepartmentID
    FROM Departments
    WHERE DepartmentName = 'HR'
```

And the `outer query` is

```
SELECT Name
FROM Employees
WHERE DepartmentID = 10;
```

Question 3: What is a transaction in SQL?

Answer:

> A **transaction** is a sequence of one or more SQL operations that run as a single execution.

It’s used to maintain the **integrity** of the database and to ensure that changes aren't applied until all queries are fully completed without errors. Otherwise, the transaction will **rollback** any changes.

What is `ACID`?

ACID represents the key properties of transactions:

- `A` for `Atomicity`: The transaction should be completed successfully or unsuccessfully, but not by parts. If any part fails, the transaction must **rollback**.
- `C` for `Consistency`: The transaction ensures data **integrity**. If any query in the transaction tries to violate a constraint, the entire transaction will **fail**, preventing changes in the database.
- `I` for `Isolation`: Ensures that transactions do not interfere with each other, even when executed simultaneously.
- `D` for `Durability`: Ensures the transaction is committed. Its change is permanently stored even in the event of a system failure.

Transactions are widely used in **Fintech**. For example, when working with clients' bank accounts, we use transactions to ensure the **integrity** of the database.

Example:
Suppose we want to transfer 100 currency units from Account 1 to Account 2. This requires two operations:

1 - Deduct 100 from Account 1's balance. 2 - Add 100 to Account 2's balance.

Here’s how our transaction query would look:

```
BEGIN;
```

```
        UPDATE Accounts
        SET Balance = Balance - 100
        WHERE AccountID = 1;

        UPDATE Accounts
        SET Balance = Balance + 100
        WHERE AccountID = 2;

        COMMIT;
```

- `BEGIN` starts the transaction
- `UPDATES` modifies the balances of the accounts.
- `COMMIT` finalizes the transaction and saves the changes, making them permanent in the database.

If any part of this process fails, we can use:

- `ROLLBACK` to undo all changes, ensuring that the database remains consistent.

Farewell & Thoughts

That's all for this post!

It’s a bit short, but I included as many examples as possible to make things easy to understand. 🫶 As I always say, if you have other questions you’d like me to cover, just let me know!

See you in the next post, and have a great week!